



Prolog

Zerina Pandža Enis Adilović Faris Aljić Nedžimija Karić

Juli 2024

Contents

1	Uvod	2
1.1	Historija Prologa	4
2	Problem 1: Planiranje i organizacija konferencije	4
3	Problem 2: Stomatološka ordinacija	11
4	Problem 3: Organizacija Taekwon-Do turnira	18
5	Problem 4: Organizacija putovanja	25
6	Izvori	31
7	Izvještaj	31

1 Uvod

Prolog (PROgramming in LOGics) je logički i deklarativni programski jezik. To je jedan od glavnih primjera jezika četvrte generacije koji podržava paradigmu deklarativnog programiranja. Ovo je posebno pogodno za programe koji uključuju simboličko ili nenumeričko računanje. Ovo je glavni razlog za korištenje Prologa kao programskog jezika u umjetnoj inteligenciji, gdje su manipulacija simbolima i manipulacija zaključivanjem osnovni zadaci.

Prije nego što krenemo objašnjavati koncept rada Prologa, potrebno je da shvatimo šta je tačno logičko programiranje.

Logičko programiranje je jedna od paradigmi kompjuterskog programiranja, u kojoj programski iskazi izražavaju činjenice i pravila o različitim problemima unutar sistema formalne logike. Ovdje su pravila napisana u obliku logičkih klauzula, gdje se nalaze glava i tijelo. Na primjer, H je glava, a B1, B2, B3 su elementi tijela. Sada, ako kažemo da je "H tačno, kada su B1, B2, B3 svi tačni", ovo je pravilo. S druge strane, činjenice su kao pravila, ali bez ikakvog tijela. Dakle, primjer činjenice je "H je istina". Za Logičko programiranje obezbijedit ćemo bazu znanja, te, koristeći ovu bazu znanja, mašina može pronaći odgovore na postavljena pitanja, što je potpuno drugačije od funkcionalnog programiranja. U funkcionalnom programiranju moramo napomenuti kako se jedan problem može riješiti, ali u logičkom programiranju moramo specificirati za koji problem zapravo želimo rješenje. Tada logičko programiranje automatski pronalazi odgovarajuće rješenje koje će nam pomoći da riješimo taj specifični problem.

Logičko programiranje	Funkcionalno programiranje
Funkcionalno programiranje prati Von-Neumannovu arhitekturu ili koristi sekvencijalne korake.	Logičko programiranje koristi apstraktni model ili se bavi objektima i njihovim odnosima.
Sintaksa je zapravo niz izjava poput (a, s, l).	Sintaksa je u osnovi logičke formule (Horn klauzule).
Računanje je uključeno tako što se naredbe izvršavaju sekvencijalno.	Računa se oduzimanjem klauzula.
Logika i kontrole su pomiješane.	Logika i kontrola se mogu razdvojiti.

Figure 1: Razlike između logičkog i funkcionalnog programiranja

U Prologu ne treba spominjati način na koji se jedan problem može riješiti, samo treba navesti koji je problem, kako bi ga Prolog automatski riješio. Međutim, u Prologu bismo trebali dati naznake kao metodu rješenja.

Jedna od ključnih karakteristika ovog programskog jezika jeste njegova sposobnost rukovanja nesigurnim ili nepotpunim informacijama. U Prologu, programer može specificirati skup pravila i činjenica za koje se zna da su istinite, ali također može specificirati pravila i činjenice koje mogu biti istinite ili netačne. Prolog

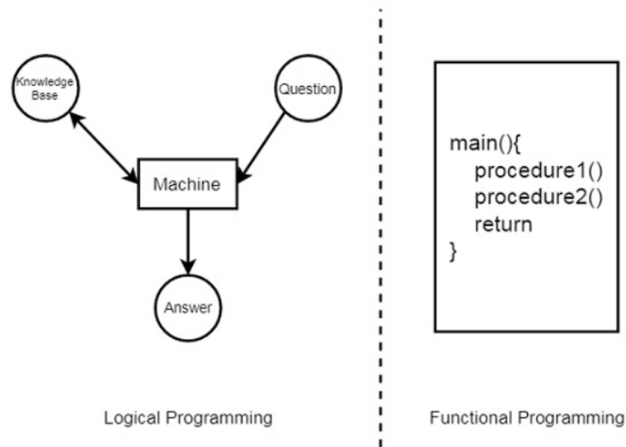


Figure 2: Razlike između logičkog i funkcionalnog programiranja prikazane putem dijagrama

interpreter će tada koristiti ta pravila i činjenice da automatski zaključi problem domene i pronađe rješenja koja će najvjerojatnije biti tačna, s obzirom na dostupne informacije.

Prolog jezik u osnovi ima tri različita elementa:

1. **Činjenice:** Činjenica je predikat koji je istinit i koriste se za pružanje informacija tumaču o domenu problema, a tumač će koristiti ove informacije da automatski zaključi rješenja problema. Naprimjer, ako kažemo, „Amar je Damirov sin“, onda je to činjenica. Pišu se pomoću imena predikata praćenog listom argumenata u zagradaama, kao naprimjer: `čovjek(Damir)`
2. **Pravila:** Pravila su logičke izjave koje opisuju odnose između različitih činjenica. U Prologu pravila se pišu korištenjem imena predikata praćenog listom argumenata u zagradaama, nakon čega slijedi dvotočka i crtica (`:-`) i tijelo pravila. Da bi se zadovoljilo pravilo, ovi uslovi moraju biti ispunjeni. Na primjer, ako definišemo pravilo kao: `djed (X, Y) :- otac (X, Z), roditelj (Z, Y)`
Ovo implicira da bi X bio djed Y, Z bi trebao biti roditelj Y i X bi trebao biti otac Z.
3. **Pitanja:** A da bismo pokrenuli prolog program, potrebna su nam neka pitanja, na koja se može odgovoriti datim činjenicama i pravilima. Oni se pišu koristeći istu sintaksu kao i činjenice praćene upitnikom (?).

Veoma važan termin koji je također potrebno spomenuti su **variable**. One se koriste za predstavljanje vrijednosti koje interpretator može promijeniti ili odrediti. Napisane su pomoću imena koje počinje velikim slovom, kao što je X ili Y.

Varijable se mogu koristiti i u činjenicama i u pravilima za predstavljanje vrijednosti koje nisu poznate u vrijeme kada je program napisan.

Da bismo koristili Prolog, morat ćete imati Prolog interpreter instaliran na vašem računalu. Na raspolaganju je nekoliko različitih tumača Prologa, uključujući *SWI-Prolog*, *GNU Prolog* i *B-Prolog*. Nakon što instaliramo interpreter, možemo početi pisati Prolog programe pomoću uređivača teksta, a zatim ih izvoditi pomoću tumača.

Ono što je bitno naglasti jeste da ne postoji jedinstvena "sintaksa" za Prolog, jer jezik dozvoljava širok spektar različitih stilova i pristupa programiranju.

1.1 Historija Prologa

Naslijeđe Prologa uključuje istraživanja dokazivača teorema i nekih drugih automatiziranih dedukcionih sistema koji su razvijeni 1960-ih i 1970-ih godina. Mehanizam zaključivanja Prologa zasniva se na Robinsonovom principu rezolucije, koji je predložen 1965. godine, i Green-ovom mehanizmu za izdvajanje odgovora (1968.). Ove ideje su se snažno spojile s pojavom procedura linearnog rješavanja.

Eksplisitne procedure linearne rezolucije usmjerene ka cilju, dale su poticaj razvoju sistema logičkog programiranja opće namjene. Prvi Prolog je bio Marseille Prolog zasnovan na radu Colmerauera iz 1970. godine. Priručnik ovog Marseille Prolog tumača (Roussel, 1975.) bio je prvi detaljan opis Prolog jezika.

Prolog se također smatra programskim jezikom četvrte generacije koji podržava paradigmu deklarativnog programiranja. Poznati japanski kompjuterski projekat pete generacije, koji je najavljen 1981. godine, usvojio je Prolog kao razvojni jezik i time privukao značajnu pažnju na jezik i njegove mogućnosti.

2 Problem 1: Planiranje i organizacija konferencije

Potrebno je organizovati trodnevnu konferenciju u Sarajevu. Na konferenciji učestvuju radnici IT firmi iz Bosne i Hercegovine, Hrvatske, Srbije i Crne Gore. Da bi konferencija bila uspješna, potrebno je organizovati govornike koji će voditi konferenciju. Od deset govornika koji su ušli u užu krug kao oni koji su dovoljno dobri da bi mogli voditi ovu konferenciju, organizator treba izabrati tri, tako da budžet može podnijeti iznos koji traži svako od govornika (dakle svaki govornik traži određeni iznos koji treba da mu se plati). Dalje, neophodno je organizovati smještaj za sve članove, osim onih koji dolaze iz Sarajeva. U opciji su Hotel Europe, Hotel Holiday i Hotel ibis Styles Sarajevo. Cijene noćenja u dvokrevetnoj sobi za jednu noć iznose: Hotel Europe 331 KM, Hotel Holiday 161 KM i Hotel ibis Styles 247 KM. Cijene noćenja u jednokrevetnoj sobi iznose: Hotel Europe 168 KM, Hotel Holiday 100 KM i Hotel ibis Styles 115. Članovi će biti raspoređeni u dvokrevetne sobe tako da u svakoj sobi budu osobe istog pola, dok će govornici dobiti jednokrevetnu sobu. Ukoliko je cijena noćena u

dvokrevetnoj sobi preko 200 KM i preko 130 KM u jednokrevetnoj sobi, organizator smatra to previsokom cijenom i odbacuje rezervaciju u tom hotelu. Pored smještaja vrlo važno je rezervisati i konferencijsku salu. Sala treba biti slobodna ta tri dana u periodima od 9:00 do 14:00, te od 16:00 do 18:00. Učesnicima je potrebno obezbijediti i majice sa natpisom koje će nositi tokom konferencije. I na kraju, kako bi organizacija bila na zavidnom nivou, potreban je dobar catering. Catering može raditi *Mrvica*, *Druga kuća*, buregdžinica *Bosna* ili *Vegeshop*. *Mrvica* nudi dobar meni po cijeni od 15 KM po osobi. *Druga kuća* nudi los meni po cijeni od 12 KM po osobi. *Bosna* nudi dobar meni po 10 KM po osobi. *Vegeshop* nudi vrhunski meni po cijeni od 18 KM po osobi. Organizator treba da izabere meni koji nije loš, ali koji će se uklopiti u trenutni budžet, tako da catering ne bi smio biti skuplji od 200 KM.

ucesnik(ime, pol, uloga, velicina majice, grad)

ucesnik("Marko", musko, govornik, "L", "Zagreb").
ucesnik("Sara", zensko, govornik, "L", "Sarajevo").
ucesnik("Aleksandar", musko, govornik, "M", "Beograd").
ucesnik("Damir", musko, govornik, "L", "Sarajevo").
ucesnik("Dijana", zensko, govornik, "M", "Sarajevo").
ucesnik("Emina", zensko, govornik, "L", "Mostar").
ucesnik("Jasmin", musko, govornik, "M", "Tuzla").
ucesnik("Irena", zensko, govornik, "XL", "Split").
ucesnik("Matija", musko, govornik, "S", "Mostar").
ucesnik("Lana", zensko, govornik, "XL", "Split").
ucesnik("Luka", musko, clan, "M", "Beograd").
ucesnik("Petra", zensko, clan, "M", "Sarajevo").
ucesnik("Tomislav", musko, clan, "XL", "Split").
ucesnik("Ivan", musko, clan, "M", "Mostar").
ucesnik("Ana", zensko, clan, "S", "Sarajevo").
ucesnik("Lejla", zensko, clan, "M", "Sarajevo").
ucesnik("Dario", musko, clan, "L", "Dubrovnik").
ucesnik("Marija", zensko, clan, "S", "Mostar").
ucesnik("Ivana", zensko, clan, "XL", "Split").
ucesnik("Josip", musko, clan, "L", "Zagreb").
ucesnik("Filip", musko, clan, "S", "Mostar").
ucesnik("Nikola", musko, clan, "M", "Tuzla").
ucesnik("Lucija", zensko, clan, "XL", "Split").
ucesnik("Tina", zensko, clan, "S", "Mostar").
ucesnik("Mario", musko, clan, "M", "Zadar").
ucesnik("Igor", musko, clan, "L", "Zagreb").
ucesnik("Kristina", zensko, clan, "M", "Podgorica").
ucesnik("Anita", zensko, clan, "L", "Zagreb").
ucesnik("Stipe", musko, clan, "M", "Tuzla").
ucesnik("Vedran", musko, clan, "L", "Sarajevo").
ucesnik("Maja", zensko, clan, "S", "Mostar").

ucesnik("Sandra", zensko, clan, "XL", "Split").
 ucesnik("Ivan", musko, clan, "L", "Zagreb").
 ucesnik("Danijela", zensko, clan, "M", "Novi Sad").
 ucesnik("Davor", musko, clan, "S", "Mostar").
 ucesnik("Nina", zensko, clan, "L", "Zagreb").
 ucesnik("Dragan", musko, clan, "M", "Tuzla").
 ucesnik("Emina", zensko, clan, "XL", "Split").
 ucesnik("Zoran", musko, clan, "L", "Sarajevo").
 ucesnik("Tea", zensko, clan, "S", "Podgorica").
 ucesnik("Lajla", zensko, clan, "S", "Cetinje").

Govornik(ime, honorar)

govornik("Marko", 1500).
 govornik("Sara", 2000).
 govornik("Aleksandar", 1200).
 govornik("Damir", 1800).
 govornik("Dijana", 1600).
 govornik("Emina", 1700).
 govornik("Jasmin", 1500).
 govornik("Irena", 1900).
 govornik("Matija", 1300).
 govornik("Lana", 1400).

raspolozivostSale(datum, pocetak, kraj).

raspolozivostSale("2024-07-10", "09:00", "14:00").
 raspolozivostSale("2024-07-10", "16:00", "18:00").
 raspolozivostSale("2024-07-11", "09:00", "14:00").
 raspolozivostSale("2024-07-11", "16:00", "18:00").
 raspolozivostSale("2024-07-12", "09:00", "14:00").
 raspolozivostSale("2024-07-12", "16:00", "18:00").

hotel(ime, cijenaDvokrevetneSobe, cijenaJednokrevetneSobe)

hotel("Hotel Europe", 331, 168).
 hotel("Hotel Holiday", 161, 100).
 hotel("Hotel ibis Styles Sarajevo", 247, 115).

catering(ime, cijenaPoOsobi, minBrojOsoba, kvalitet)

catering("Mrvica", 15, 50, dobar).
 catering("Druga kuća", 12, 30, los).
 catering("Buregdžinica Bosna", 10, 20, dobar).
 catering("Vegehop", 18, 10, vrhunski).

Prihvatljiv hotel

prihvatljivHotel(Ime, Dvokrevetne, Jednokrevetne) :-
 hotel(Ime, CijenaDvokrevetne, CijenaJednokrevetne),
 CijenaDvokrevetne =< 200, CijenaJednokrevetne =< 130,
 Dvokrevetne = CijenaDvokrevetne,
 Jednokrevetne = CijenaJednokrevetne.

Upit kojim tražimo najprihvatljiviji hotel:

?- prihvatljivHotel(Ime, CijenaDvokrevetne, CijenaJednokrevetne).

Broj učesnika koji nisu iz Sarajeva i koji trebaju smještaj

brojClanova(24).

Odabir govornika shodno mogućnostima

(Generiše sve kombinacije od K elemenata liste L)

combinations(0, _, []). combinations(K, [H|T], [H|Comb]) :- K > 0, K1 is K - 1, combinations(K1, T, Comb). combinations(K, [_|T], Comb) :- K > 0, combinations(K, T, Comb).

izaberiGovornike(SelektovaniGovornici, Budzet) :- findall(govornik(Ime, Cijena), govornik(Ime, Cijena), Govornici), combinations(3, Govornici, SelektovaniGovornici), ukupanTrosak(SelektovaniGovornici, Ukupno), Ukupno =< Budzet.

ukupanTrosak([], 0). ukupanTrosak([govornik(_, Cijena)|Ostali], Ukupno):-
ukupanTrosak(Ostali, Trosak), Ukupno is Trosak + Cijena.

Upit kojim tražimo tri govornika za konferenciju:

?- izaberiGovornike(SelektovaniGovornici, 4000).

Izbor hotela i raspodjela soba

raspodjelaSoba(Hotel, BrojJednokrevetnih, BrojDvokrevetnih, Budzet) :-
brojClanova(N), BrojJednokrevetnih is 3, BrojDvokrevetnih is ceil(N / 2),
izaberiGovornike(SelektovaniGovornici, Budzet),
prihvatljivHotel(Hotel, CijenaDvokrevetne, CijenaJednokrevetne),
UkupniTrosak is CijenaDvokrevetne * BrojDvokrevetnih + CijenaJednokrevetne
* BrojJednokrevetnih, UkupniTrosak =< Budzet.

Testiranje sistema:

?- raspodjelaSoba(Hotel, BrojJednokrevetnih, BrojDvokrevetnih, 5000).

Izbor cateringa prema budžetu i broju učesnika

izborCatering(Ime, BrojUcesnika, Budzet) :- catering(Ime, CijenaPoOsobi,
MinBrojOsoba, Kvalitet), Kvalitet \= los, BrojUcesnika >= MinBrojOsoba, UkupnaCijena is BrojUcesnika * CijenaPoOsobi, UkupnaCijena =< Budzet.

Upit kojim biramo najprikladniju ponudu za catering:

?- izborCatering(Ime, 100, 1200).

Dodavanje opcija za članove koji konzumiraju posebnu dijetu

catering("Vegehop", 18, 10, vrhunski, veganski).

Prošireno pravilo koje uključuje dijetetske preference

izborCateringPoDijeti(Ime, BrojUcesnika, Budzet, Dijeta) :- catering(Ime,
CijenaPoOsobi, MinBrojOsoba, Kvalitet, Dijeta), Kvalitet \= los, BrojUcesnika >=

$MinBrojOsoba, UkupnaCijena \leq BrojUcesnika * CijenaPoOsobi, UkupnaCijena = < Budzet.$

Provjera da li je sala dostupna za navedene datume

provjeriDostupnost(Datum) :- raspolozivostSale(Datum, "09:00", "14:00"),
raspolozivostSale(Datum, "16:00", "18:00").

Provjera za sve datume konferencije

provjeraKonferencije(ListaDatuma) :- maplist(provjeriDostupnost, ListaDatuma).

Planiranje konferencije

planirajKonferenciju(ListaDatuma) :-
provjeraKonferencije(ListaDatuma), writeln('Sala je dostupna za sve navedene datume i vremena.').

Upit za provjeru plana konferencije:

?- planirajKonferenciju(["2024-07-10", "2024-07-11", "2024-07-12"]).

Kolekcioniranje svih veličina majica i njihovih brojeva

velicinaMajice(Velicina, Broj) :-
findall(V, ucesnik(_, _, _, V, _), Velicine), member(Velicina, Velicine),
aggregate(count, member(Velicina, Velicine), Broj).

Izlistavanje potrebnih majica po veličinama

izlistajMajice :-
velicinaMajice(Velicina, Broj), format('Potrebno je d majica veličine w.',
[Broj, Velicina]), fail.
izlistajMajice.

Upit za islistavanje majica:

?- izlistajMajice.

```

izbor_catering(Ime, 100, 1200).
Singleton variables: [G1,G2,G3,N,Budzet]
Ime = "Druga kuća"
Ime = "Buregdžinica Bosna"
false

planiraj_konferenciju(["2024-07-10", "2024-07-11", "2024-07-12"]).
Singleton variables: [G1,G2,G3,N,Budzet]
Sala je dostupna za sve navedene datume i vremena.
true
false

prihvatljiv_hotel(Ime, CijenaDvokrevetne, CijenaJednokrevetne).
Singleton variables: [G1,G2,G3,N,Budzet]
CijenaDvokrevetne = 161,
CijenaJednokrevetne = 100,
Ime = "Hotel Holiday"
false

izaberi_govornike(SelektovaniGovornici, 4000).
Singleton variables: [G1,G2,G3,N,Budzet]
SelektovaniGovornici = [govornik("Marko",1500), govornik("Aleksandar",1200), govornik("Matija",1300)]
SelektovaniGovornici = [govornik("Aleksandar",1200), govornik("Jasmin",1500), govornik("Matija",1300)]
SelektovaniGovornici = [govornik("Aleksandar",1200), govornik("Matija",1300), govornik("Lana",1400)]
false

```

Figure 3: Izlaz programa za upite: izborCatering(Ime, 100, 1200)., planirajKonferenciju(["2024-07-10", "2024-07-11", "2024-07-12"]), prihvatljivHotel(Ime, CijenaDvokrevetne, CijenaJednokrevetne) i izaberiGovornike(SelektovaniGovornici, 4000).

```

raspodjela_soba(Hotel, BrojJednokrevetnih, BrojDvokrevetnih, 5000).
Singleton variables: [SelektovaniGovornici]
BrojDvokrevetnih = 12,
BrojJednokrevetnih = 3,
Hotel = "Hotel Holiday"
BrojDvokrevetnih = 12,
BrojJednokrevetnih = 3,
Hotel = "Hotel Holiday"
BrojDvokrevetnih = 12,
BrojJednokrevetnih = 3,
Hotel = "Hotel Holiday"

izlistaj_majice
Singleton variables: [SelektovaniGovornici]
Potrebno je 12 majica veličine L.
Potrebno je 12 majica veličine L.
Potrebno je 13 majica veličine M.
Potrebno je 12 majica veličine L.
Potrebno je 13 majica veličine M.
Potrebno je 12 majica veličine L.
Potrebno je 13 majica veličine M.
Potrebno je 7 majica veličine XL.
Potrebno je 9 majica veličine S.
Potrebno je 7 majica veličine XL.
Potrebno je 13 majica veličine M.
Potrebno je 13 majica veličine M.
Potrebno je 7 majica veličine XL.
Potrebno je 13 majica veličine M.
Potrebno je 8 majica veličine S.

```

Figure 4: Izlaz programa za upite: raspodjelaSoba(Hotel, BrojJednokrevetnih, BrojDvokrevetnih, 5000) i izlistajMajice.

Zaključak:

Uz pomoć datog koda u Prologu smo uspjeli organizirati konferenciju, pri čemu su ispunjeni svi zahtjevi organizatora (iz samog izlaza programa možemo vidjeti konačni rezultat). Ukoliko bi organizator želio planirati neke naredne konferencije sa istim zahtjevima, bilo bi potrebno definisati nove činjenice, jer zasigurno neće imati isti broj članova (muških ili ženskih) i slično, što znači da ovaj kod nije u potpunosti univerzalan i bilo kakvi drugačiji zahtjevi bi za sobom povukli neophodnu izmjenu postojećeg koda. Uz neophodne izmjene, program bi mogao biti unaprijeđen na način da u istoj sobi budu osobe iz iste firme koje se već poznaju, te da dnevne aktivnosti budu bolje organizovane i konkretnije objašnjene.

3 Problem 2: Stomatološka ordinacija

Zamislamo da nam je potreban termin u stomatološkoj ordinaciji u kojoj radi n stomatologa i svaki od njih ima svoje radno vrijeme. U stanju smo birati termin koji nama odgovara, ali također moramo uzeti u obzir da taj termin ne odgovara svim stomatolozima u ordinaciji, neki od njih u tom terminu možda nisu na smjeni, dok je kod nekih eventualno taj termin popunjen. Uz kreiranje smislene baze podataka i određenih pravila, prolog je mehanizam koji nam je u stanju pružiti savršeno rješenje našeg problema, ako ono postoji.

Broj stomatologa je potpuno fleksibilan i ništa nas ne sprečava da dodamo novog ili uklonimo nekog od trenutnih stomatologa, shodno situaciji. U našem slučaju, da bismo pokazali osnovne funkcionalnosti zamišljenog programa, dovoljna su nam dva stomatologa, koja možemo kreirati kao činjenice, i to na sljedeći način:

zubar(z1).
zubar(z2).

Radi dodatne fleksibilnosti, kreirat ćemo i činjenice koje će predstavljati radno vrijeme za svakog od stomatologa. One nisu fiksne i možemo ih promijeniti kad god nam to zatreba. U njima ćemo čuvati tri podatka: ime stomatologa ($z1$ i $z2$ u našem slučaju), te početak i kraj njegovog radnog vremena.

radno_vrijeme(z1, 8, 16).
radno_vrijeme(z2, 16, 24).

Nakon što smo u našu bazu podataka smjestili stomatologe i njihova radna vremena, sada možemo preći na kreiranje termina. Termin će biti činjenica koja redom sadrži sljedeće: ime stomatologa kod kojeg je zakazan, datum, vrijeme, informaciju o tome da li je popunjen ili slobodan, te peto polje koje je u slučaju slobodnog termina prazno, a u slučaju popunjenog termina sadrži ime pacijenta za taj termin. Za potrebe ove zadaće, pretpostavit ćemo da jedan termin kod stomatologa traje fiksno sat vremena. Kreirajmo termine jednog radnog dana za ova dva stomatologa:

: -dynamic termin/5.
termin(z1, [1, 1, 2025], 8, popunjeno, mujo).
termin(z1, [1, 1, 2025], 9, popunjeno, fata).
termin(z1, [1, 1, 2025], 10, slobodno,).
termin(z1, [1, 1, 2025], 11, slobodno,).
termin(z1, [1, 1, 2025], 12, popunjeno, suljo).
termin(z1, [1, 1, 2025], 13, slobodno,).
termin(z1, [1, 1, 2025], 14, slobodno,).
termin(z1, [1, 1, 2025], 15, slobodno,).
termin(z2, [1, 1, 2025], 16, slobodno,).

```

termin(z2, [1, 1, 2025], 17, slobodno, ).
termin(z2, [1, 1, 2025], 18, popunjeno, haso).
termin(z2, [1, 1, 2025], 19, slobodno, ).
termin(z2, [1, 1, 2025], 20, slobodno, ).
termin(z2, [1, 1, 2025], 21, slobodno, ).
termin(z2, [1, 1, 2025], 22, popunjeno, huso).
termin(z2, [1, 1, 2025], 23, slobodno, ).

```

Dio koda: ":- dynamic termin/5." omogućava nam kreiranje dinamičkog predikata "termin" sa 5 polja, odnosno omogućava nam vršenje izmjena u ovako kreiranim predikatima.

Sada u našoj maloj bazi podataka imamo sasvim dovoljno činjenica da ispitamo sve funkcionalnosti našeg programa koje su nam potrebne.

Za početak, najbolje bi bilo da provjerimo ispravnost datuma i vremena u terminu koji obrađujemo. Za to će nam služiti sljedeća pravila:

```

prestupna_godina(Godina) : -
    (Godina mod 4 =:= 0, Godina mod 100 = \ = 0);
    (Godina mod 400 =:= 0).

broj_dana(1, _, 31). %Januar
broj_dana(2, Godina, D) : - prestupna_godina(Godina) -> D = 29;
D = 28. %Februar
broj_dana(3, _, 31). %Mart
broj_dana(4, _, 30). %April
broj_dana(5, _, 31). %Maj
broj_dana(6, _, 30). %Juni
broj_dana(7, _, 31). %Juli
broj_dana(8, _, 31). %August
broj_dana(9, _, 30). %Septembar
broj_dana(10, _, 31). %Oktobar
broj_dana(11, _, 30). %Novembar
broj_dana(12, _, 31). %Decembar

ispravan_datum([Dan, Mjesec, Godina]) : -
    Mjesec >= 1, Mjesec <= 12,
    Godina > 0,
    broj_dana(Mjesec, Godina, MaxDan),
    Dan >= 1, Dan <= MaxDan.

ispravno_vrijeme(Zubar, Sat) : -
    Pocetak >= 0, Pocetak <= 23,
    Kraj >= 1, Kraj <= 24,
    radno_vrijeme(Zubar, Pocetak, Kraj),
    Sat >= Pocetak, Sat < Kraj.

```

Kreirane funkcionalnosti za provjeru ispravnosti datuma i vremena možemo koristiti u ostalim pravilima koja ćemo koristiti za manipulaciju terminima.

Prvo od tih pravila ispituje da li je neki termin dostupan za rezervaciju. Odgovor je "da", odnosno "true" ako su ispravni datum i vrijeme te ako četvrto polje u razmatranom terminu ima vrijednost "slobodno". Ako je neispravan datum ili vrijeme, ako smo zadali nepostojeće ime stomatologa ili ako je sve to validno ali polje koje nam daje status termina nema vrijednost "slobodno", ovo pravilo će nam na svom izlazu vratiti vrijednost "false".

```
dostupan(Zubar, [Dan, Mjesec, Godina], Sat) : -
    ispravan_datum([Dan, Mjesec, Godina]),
    ispravno_vrijeme(Zubar, Sat),
    termin(Zubar, [Dan, Mjesec, Godina], Sat, slobodno, _).
```

Sad kad imamo sve ove funkcionalnosti, možemo kreirati pravila za rezervaciju i otkazivanje termina. Ovdje do izražaja dolazi kreiranje termina kao dinamičkih predikata iz razloga što ćemo morati mijenjati posljednja dva polja termina.

Predikat rezervisi_termin provjerava da li je razmatrani termin dostupan, a na taj način automatski je provjerena i ispravnost datuma i vremena. Ako je termin dostupan, mijenja se vrijednost predzadnjeg polja na "popunjeno", a zadnjem polju dodjeljujemo ime pacijenta koji rezerviše taj termin.

```
rezervisi_termin(Zubar, [D, M, G], Sat, Pacijent) : -
    dostupan(Zubar, [D, M, G], Sat),
    termin(Zubar, [D, M, G], Sat, slobodno, _),
    retract(termin(Zubar, [D, M, G], Sat, slobodno, _)),
    assert(termin(Zubar, [D, M, G], Sat, popunjeno, Pacijent)).
```

Kod predikata otkazi_termin moramo prije svega provjeriti ispravnost datuma i vremena, a nakon toga provjeravamo da li je termin popunjen. Ako jeste, vrijednost predzadnjeg polja se mijenja sa "popunjeno" na "slobodno", a posljednje polje predviđeno za ime pacijenta nam nije od naročitog značaja. Ako je termin već slobodan, takav će i ostati, bez ikakvih promjena.

```
otkazi_termin(Zubar, [Dan, Mjesec, Godina], Sat) : -
    ispravan_datum([Dan, Mjesec, Godina]),
    ispravno_vrijeme(Zubar, Sat),
    termin(Zubar, [Dan, Mjesec, Godina], Sat, popunjeno, _),
    retract(termin(Zubar, [Dan, Mjesec, Godina], Sat, popunjeno, _)),
    assert(termin(Zubar, [Dan, Mjesec, Godina], Sat, slobodno, _)).
```

U realnom svijetu, kako za stomatologe, tako i za pacijente, bilo bi jako korisno da možemo na neki jednostavan način izlistati sve dostupne stomatologe, kao i sve slobodne i zauzete termine. U prologu to možemo jednostavno uraditi na način da od njih kreiramo listu. U suštini, to će nam odmah omogućiti i njihov prikaz, ali bit će puno preglednije ako posebno napravimo predikate za ispis kreiranih listi kako bi to bilo znatno preglednije.

```
svi_zubari(Zubari) : -
    findall(Zubar, zubar(Zubar), Zubari).
```

```
svi_slobodni_termini(Slobodni) : -
    findall(
        [Zubar, D, M, G, H],
        (zubar(Zubar), termin(Zubar, [D, M, G], H, slobodno, _)),
        Slobodni
    ).
```

```
svi_zauzeti_termini(Zauzeti) : -
    findall(
        [Zubar, D, M, G, H, Pacijent],
        (zubar(Zubar), termin(Zubar, [D, M, G], H, popunjeno, Pacijent)),
        Zauzeti
    ).
```

```
ispisi_slobodne_termini : -
    svi_slobodni_termini(Slobodni),
    forall(
        member([Zubar, D, M, G, H], Slobodni),
        (format('Zubar : ~ wDatum : ~ d. ~ d. ~ d. Vrijeme : ~ d ~
n', [Zubar, D, M, G, H]))
    ).
```

```
ispisi_zauzete_termini : -
    svi_zauzeti_termini(Zauzeti),
    forall(
        member([Zubar, D, M, G, H, Pacijent], Zauzeti),
        (format('Zubar : ~ wDatum : ~ d. ~ d. ~ d. Vrijeme : ~ dPacijent :
~ w ~ n', [Zubar, D, M, G, H, Pacijent]))
    ).
```

Sve prethodno implemetirane funkcionalnosti možemo prikazati kroz nekoliko upita. Za početak, najbolje bi bilo da se uvjerimo u ispravno funkcionisanje provjere ispravnosti datuma i vremena, iz razloga što se na ove predikate oslanjaju svi ostali.

```

?- ispravan_datum([29, 2, 2020]).
true.
?- ispravan_datum([29, 2, 2021]).
false.
?- ispravan_datum([29, 2, 2000]).
true.
?- ispravan_datum([29, 2, 1900]).
false.
?- ispravan_datum([31, 1, 2021]).
true.

```

Figure 5: Provjera ispravnosti datuma, uključujući granične slučajeve za prestupnu godinu

Da bismo u potpunosti korektno ispitali ispravnost vremena, potrebno je da saznamo radno vrijeme kreiranih zubara, a to ćemo postići sljedećim upitima:

```

?- zubar(z1).
true.
?- zubar(z2).
true.
?- radno_vrijeme(z1, Pocetak, Kraj).
Pocetak = 8,
Kraj = 16.
?- radno_vrijeme(z2, Pocetak, Kraj).
Pocetak = 16,
Kraj = 24.

```

Figure 6: Provjera radnog vremena za kreirane zubare

Predikat `ispravno_vrijeme`, osim što provjerava da li je termin u korektnom opsegu 0-23, također provjerava i da li je on u opsegu korektnom sa aspekta radnog vremena razmatranog zubara.

```

?- ispravno_vrijeme(z1, 8).
true.
?- ispravno_vrijeme(z1, 16).
false.
?- ispravno_vrijeme(z1, 15).
true.
?- ispravno_vrijeme(z2, 15).
false.
?- ispravno_vrijeme(z2, 16).
true.
?- ispravno_vrijeme(z2, 23).
true.

```

Figure 7: Provjera ispravnosti vremena


```

?- dostupan(z1, [1, 1, 2025], 8).
false.
?- dostupan(z1, [1, 1, 2025], 10).
true.
?- dostupan(z1, [1, 1, 2025], 16).
false.
?- dostupan(z2, [1, 1, 2025], 16).
true.
?- dostupan(z2, [1, 1, 2025], 22).
false.

```

Figure 8: Provjera dostupnosti zubara u željenom terminu

Zaključak:

Nakon ispitivanja ovih osnovnih funkcionalnosti, možemo preći na predikate vezane za manipulaciju našom bazom podataka. Za početak, najbolje bi bilo da ispišemo sve termine koje imamo u našoj bazi. Ispisat ćemo prvo slobodne i zauzete termine, zatim izvršiti nekoliko otkazivanja i novih rezervacija, te na kraju ponovo ispisati modificirani raspored termina kako bismo se zaista uvjerali u ispravnost rezervacija i otkazivanja.

```

?- svi_zubari(Zubari).
Zubari = [z1, z2].

?- ispisi_slobodne_termine().
Zubar: z1 Datum: 1.1.2025. Vrijeme: 10
Zubar: z1 Datum: 1.1.2025. Vrijeme: 11
Zubar: z1 Datum: 1.1.2025. Vrijeme: 13
Zubar: z1 Datum: 1.1.2025. Vrijeme: 14
Zubar: z1 Datum: 1.1.2025. Vrijeme: 15
Zubar: z2 Datum: 1.1.2025. Vrijeme: 16
Zubar: z2 Datum: 1.1.2025. Vrijeme: 17
Zubar: z2 Datum: 1.1.2025. Vrijeme: 19
Zubar: z2 Datum: 1.1.2025. Vrijeme: 20
Zubar: z2 Datum: 1.1.2025. Vrijeme: 21
Zubar: z2 Datum: 1.1.2025. Vrijeme: 23
true.

?- ispisi_zauzete_termine().
Zubar: z1 Datum: 1.1.2025. Vrijeme: 8 Pacijent: mujo
Zubar: z1 Datum: 1.1.2025. Vrijeme: 9 Pacijent: fata
Zubar: z1 Datum: 1.1.2025. Vrijeme: 12 Pacijent: suljo
Zubar: z2 Datum: 1.1.2025. Vrijeme: 18 Pacijent: haso
Zubar: z2 Datum: 1.1.2025. Vrijeme: 22 Pacijent: huso
true.

```

Figure 9: Dostupni zubari i raspored njihovih slobodnih i zauzetih termina

```

?- otkazi_termin(z2, [1, 1, 2025], 18).
true.
?- otkazi_termin(z2, [1, 1, 2025], 22).
true.
?- otkazi_termin(z1, [1, 1, 2025], 8).
true.
?- otkazi_termin(z1, [1, 1, 2025], 9).
true.

```

Figure 10: Otkazivanja

```

?- rezervisi_termin(z1, [1, 1, 2025], 13, haso).
true.

?- rezervisi_termin(z2, [1, 1, 2025], 16, huso).
true.

?- rezervisi_termin(z2, [1, 1, 2025], 22, fata).
true.

?- rezervisi_termin(z2, [1, 1, 2025], 23, mujo).
true.

```

Figure 11: Nove rezervacije

```

?- ispisi_slobodne_termine().
Zubar: z1 Datum: 1.1.2025. Vrijeme: 10
Zubar: z1 Datum: 1.1.2025. Vrijeme: 11
Zubar: z1 Datum: 1.1.2025. Vrijeme: 14
Zubar: z1 Datum: 1.1.2025. Vrijeme: 15
Zubar: z1 Datum: 1.1.2025. Vrijeme: 8
Zubar: z1 Datum: 1.1.2025. Vrijeme: 9
Zubar: z2 Datum: 1.1.2025. Vrijeme: 17
Zubar: z2 Datum: 1.1.2025. Vrijeme: 19
Zubar: z2 Datum: 1.1.2025. Vrijeme: 20
Zubar: z2 Datum: 1.1.2025. Vrijeme: 21
Zubar: z2 Datum: 1.1.2025. Vrijeme: 18
true.

?- ispisi_zauzete_termine().
Zubar: z1 Datum: 1.1.2025. Vrijeme: 12 Pacijent: suljo
Zubar: z1 Datum: 1.1.2025. Vrijeme: 13 Pacijent: haso
Zubar: z2 Datum: 1.1.2025. Vrijeme: 16 Pacijent: huso
Zubar: z2 Datum: 1.1.2025. Vrijeme: 22 Pacijent: fata
Zubar: z2 Datum: 1.1.2025. Vrijeme: 23 Pacijent: mujo
true.

```

Figure 12: Raspored slobodnih i zauzetih termina nakon izvršenih izmjena

Uvjerili smo se da sve naše funkcionalnosti izvedene u ovom programu zaista rade korektno, ali sve ovo je svedeno na neki najosnovniji minimum potreban za rad sa problemom ovolikih razmjera. Bilo bi poprilično neefikasno da svaki termin i svaki radni dan moramo ručno kreirati. Tim povodom, u budućnosti bi bilo jako korisno omogućiti automatsko generisanje termina, npr. prolaskom kroz petlju između dva datuma te za svaki datum od početka do kraja radnog vremena. Osim toga, primijetimo da je u našem primjeru upotrijebljen datum 1.1.2025., a šanse da ćemo naći nekog stomatologa koji zaista radi prvog januara su jako male. I to možemo uzeti u obzir te odrediti sve neradne dane koje bismo preskočili prilikom generisanja termina. Vidimo da postoji more dodatnih mogućnosti i njihova implementacija bi najvjerojatnije zahtijevala promjenu postojećeg koda, ali s obzirom da smo se uz ovaj primjer upoznali sa osnovnim funkcionalnostima prologa, prelazak na neki viši nivo rada sa prologom bi za sigurno bio znatno olakšan.

4 Problem 3: Organizacija Taekwon-Do turnira

Organizacija Taekwon-Do turnira je veliki izazov u kojem je potrebno pažljivo planiranje i koordinisanje. Naravno, treba se baviti različitim aspektima događaja kao što su, na primjer, prijavljivanje takmičara, raspoređivanje mečeva i formi, pauze za ručak i na kraju dodjeljivanje nagrada. Da sve te aktivnosti budu u skladu kako bi se obezbijedila uspješnost turnira bez ikakvih odstupanja.

Prvo ćemo definisati broj takmičara. Međutim ovdje nemamo nikakvo ograničenje, broj takmičara se može mijenjati.

Definisanje takmičara:

takmicar(ime _prezime, godine, tezina, pojas).
takmicar("IvanIvanovi", 10, -30, zuti _pojas).
takmicar("MarkoMarkovi", 12, -35, zeleni _pojas).
takmicar("PetarPetrovi", 14, -40, plavi _pojas).
takmicar("AnaAni", 16, -50, crni _pojas).
takmicar("SaraSari", 17, -55, crni _pojas).
takmicar("EmaEmi", 20, -60, crni _pojas).
takmicar("LukaLuki", 22, -70, crni _pojas).
takmicar("AdnanAdnanovi", 23, -68, crni _pojas).

Nakon što smo definisali takmicare, na osnovu njihovih bitnih karakteristika, sada se trebamo fokusirati na daljnji razvoj disciplina koje će organizator turnira ponuditi. Sada je potrebno odrediti moguće discipline.

Definisanje disciplina:

disciplina(borbe _pojedinačno).
disciplina(forme _pojedinačno).
disciplina(borbe _ ekipno).

Postoje četiri kategorije na osnovu starosti takmičara: *mladji _kadet*, *stariji _kadet*, *junior* i *senior*. Svaka kategorija ima svoju maksimalnu starosnu granicu. *Starosna _kategorija* je predikat koji za dato ime takmičara pronalazi odgovarajuću kategoriju starosne dobi na osnovu godina takmičara i definisanih kategorija.

Definisanje kategorija starosne dobi:

kategorija(mladji _kadet, 12).
kategorija(stariji _kadet, 14).
kategorija(junior, 18).
kategorija(senior, 45).

Zadali smo kategorije po starosnoj dobi, nastojimo svakom takmičaru pronaći njegovu odgovarajuću kategoriju.

Pronalaženje kategorije starosne dobi za takmičara:

```
starosna_kategorija(Ime, Kategorija) : –
takmicar(Ime, Godine, _, _),
kategorija(Kategorija, MaxGodine),
Godine =< MaxGodine,
(kategorija(_, DrugaMaxGodine), DrugaMaxGodine < MaxGodine, Godine =<
DruaMaxGodine).
```

Predpostavimo da nakon svih izvršnih postupaka takmičenje je krenulo svojim tokom. Te na kraju takmičenja treba definisati rezultate unutar svake kategorije.

Definisanje rezultata:

```
rezultat(ime, disciplina, kategorija, pozicija).
rezultat("Ivan", borbe_pojedinacno, -30, 1).
rezultat("Marko", borbe_pojedinacno, -35, 2).
rezultat("Petar", borbe_pojedinacno, -40, 3).
rezultat("Ana", forme_pojedinacno, crni_pojas, 1).
rezultat("Sara", forme_pojedinacno, crni_pojas, 2).
rezultat("Ema", forme_pojedinacno, crni_pojas, 3).
```

Nakon što smo pohranili podatke o rezultatima održanog takmičenja, natojimo pronaći najbolje takmičare na turniru kako bismo dodijelili nagrade. Najbolji_borac pronalazi takmičare koji su osvojili prvo mjesto u svojoj starosnoj kategoriji u nekoj disciplini.

Pronalaženje najboljih boraca po kategorijama: *najbolji_borac(Ime, Kategorija) :*

```
– starosna_kategorija(Ime, Kategorija), rezultat(Ime, _, _, 1).
```

Sada ćemo na osnovu prikupljenih podataka sve najbolje smjestiti na jedno mjesto, radi lakšeg kreiranja liste najboljih boraca. Najbolji_borci pronalazi takmičare koji su bili najbolji u pojedinačnim disciplinama.

Pronalaženje svih najboljih boraca

```
najbolji_borci(Borci) : –
findall((Ime, Kategorija), najbolji_borac(Ime, Kategorija), Borci).
```

Konačno bismo trebali prikazati najbolje i njima uručiti nagrade. Prikazi_najbolje_borce funkcija prikazuje sve najbolje borce po kategorijama.

Prikazivanje najboljih boraca

```
prikazi_najbolje_borce : –
najbolji_borci(Borci),
forall(member((Ime, Kategorija), Borci),
format('Najbolji borac u kategoriji ~ w : ~ w ~ n', [Kategorija, Ime])).
```

Planiraj_turnir je glavni predikat koji prikazuje plan turnira u skladu sa definisanim disciplinama i vremenima održavanja. U ovom slučaju, prikazuje se raspored od početka do kraja turnira sa vremenskim okvirima za svaku fazu (forme pojedinačno, borbe pojedinačno, borbe ekipno, pauza za ručak, dodjela medalja). Radi lakšeg određivanja vremenskog intervala održavanja turnira pretpostaviti ćemo da su sve borbe trajale jednako bez odstupanja.

Planiranje turnira sa pauzom

```
planiraj_turnir : -
writeln('Poetaktturnira : 09 : 00'),
writeln('Formepojedinano : 09 : 00 - 13 : 00'),
writeln('Pauzazaruak : 13 : 00 - 15 : 00'),
writeln('Borbepojedinano : 15 : 00 - 20 : 00'),
writeln('Borbeekipno : 20 : 00 - 21 : 00'),
writeln('Dodjelamedaljaipehara : 21 : 00 - 21 : 30'),
prikazi_najbolje_borce,
writeln('Krajturnira : 21 : 30').
```

Sada ćemo testirati svaku funkcionalnost koju smo naveli prethodno. Ovdje ćemo postaviti uslove:

```
iznad_bijelog_pojava(zuti_pojava).
iznad_bijelog_pojava(zeleni_pojava).
iznad_bijelog_pojava(plavi_pojava).
iznad_bijelog_pojava(crni_pojava).

testiraj_takmicara(Ime) : -
takmicar(Ime, Godine, Tezina, Pojava),
Godine > 5,
iznad_bijelog_pojava(Pojava),
format('Takmiar :~ w, Godine :~ w, Teina :~ w, Pojava :~ w ~ n', [Ime, Godine, Tezina, Pojava]).
```

Sa ovim zadanim uslovima ćemo testirati unos takmičara. Takmičar mora imati veće znanje od bijelog pojasa, kao jedan od ključnih uslova.

```
% Definisanje takmičara
takmicar("Ivan Ivanović", 10, -30, bijeli_pojas).
takmicar("Marko Marković", 12, -35, zeleni_pojas).
takmicar("Petar Petrović", 14, -40, ljubičasti_pojas).
```

Figure 13: Unesesni takmičari

```
testiraj_takmicara("Ivan Ivanović").
false
testiraj_takmicara("Marko Marković").
Takmičar: Marko Marković, Godine: 12, Težina: -35, Pojas: zeleni_pojas
true
testiraj_takmicara("Petar Petrović").
false
```

Figure 14: Rezultati testiranja

Nakon što smo testirali predikat `testiraj_takmicara`, možemo zaključiti da ispravno funkcioniše. Za takmičare koji su imali zvanje bijeli pojas i zvanje koje ne postoji ti takmičari su detektovani.

Sada ćemo testirati da li se takmičar nalazi u dvije starosne kategorije. Ovim testom ćemo spriječiti grešku pri unosu jer takmičar ne može biti istovremeno i 20 godina i 10 godina star.

testiraj_starosnu_kategoriju(Ime) : –
starosna_kategorija(Ime, Kategorija),
format('Takmiar :~ w, Starosna_kategorija :~ w ~ n', [Ime, Kategorija]),
\+(starosna_kategorija(Ime, DrugaKategorija), Kategorija \= DrugaKategorija),

format('Takmiar ~ w se ne nalazi u dvije razliite starosne kategorije. ~
n', [Ime]).

```
% Definisanje takmičara
takmicar("Ana Anić", 16, -50, crni_pojas).
takmicar("Sara Sarić", 17, -55, crni_pojas).
takmicar("Sara Sarić", 19, -55, crni_pojas).
```

Figure 15: Uneseni takmičari

```

testiraj_starosnu_kategoriju("Sara Sarić").
Takmičar: Sara Sarić, Starosna kategorija: junior
Takmičar: Sara Sarić, Starosna kategorija: senior
false

testiraj_starosnu_kategoriju("Ana Anić").
Takmičar: Ana Anić, Starosna kategorija: junior
Takmičar Ana Anić se ne nalazi u dve različite starosne kategorije.
true

```

Figure 16: Rezultati testiranja

Predikat za testiranje starosne kategorije radi i u slučaju da se ista osoba pojavljuje u dvije različite kategorije.

Sada je vrijeme da ispitamo predikat za upis rezultata. To ćemo učiniti na slijedećin način.

*testiraj_rezultat(Ime, Disciplina, Kategorija, Rang) : –
 rezultat(Ime, Disciplina, Kategorija, Rang),
 \+(rezultat(Ime, Disciplina, Kategorija, DrugiRang), DrugiRang \= Rang),
 format('Rezultat–Takmicar :~ w, Disciplina :~ w, Kategorija :~ w, Rang :~
 w ~ n', [Ime, Disciplina, Kategorija, Rang]).*

```

% Definisane rezultata
rezultat("Marko Marković", borbe_pojedinačno, -35, 2).
rezultat("Petar Petrović", borbe_pojedinačno, -40, 3).
rezultat("Ema Emić", forme_pojedinačno, crni_pojas, 3).
rezultat("Ema Emić", forme_pojedinačno, crni_pojas, 1).

```

Figure 17: Unos rezultata

```

testiraj_rezultat("Marko Marković", borbe_pojedinačno, -35, 2).
Rezultat - Takmičar: Marko Marković, Disciplina: borbe_pojedinačno, Kategorija: -35, Rang: 2
true

testiraj_rezultat("Petar Petrović", borbe_pojedinačno, -60, 3).
false

testiraj_rezultat("Ema Emić", forme_pojedinačno, crni_pojas, 3).
false

```

Figure 18: Rezultati testiranja

Nakon što smo testirali i predikat `testiraj_rezultat` i ustanovili da nema takmičara koji imaju dva rezultata u istoj kategoriji, također i da ne mogu imati rezultate u drugim kategorijama koje njima ne pripadaju. Možemo ispitati neke osnovne predikate.

Ovim predikatom ćemo ispitati da li postoje najbolji borci te da ih prikazemo.

```
testiraj_najbolje_borce : -
najbolji_borci(Borci),
format('Najboljiborci : , ~ w ~ n', [Borci]).
```

```
testiraj_prikaz_najboljih_boraca : -
prikazi_najbolje_borce.
```

```
% Definisanje rezultata
rezultat("Ivan Ivanović", borbe_pojedinačno, -30, 1).
rezultat("Marko Marković", borbe_pojedinačno, -35, 2).
rezultat("Petar Petrović", borbe_pojedinačno, -40, 3).
rezultat("Ana Anić", forme_pojedinačno, crni_pojas, 1).
rezultat("Sara Sarić", forme_pojedinačno, crni_pojas, 2).
rezultat("Ema Emić", forme_pojedinačno, crni_pojas, 3).
```

Figure 19: Definisani rezultati

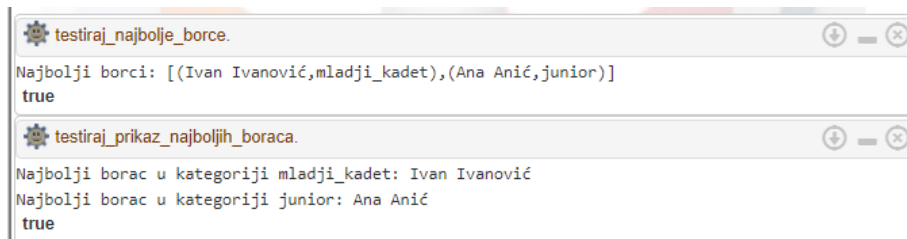


Figure 20: Pronađeni najbolji borci i njihov ispis

Na kraju ćemo pozvati predikat koji ispisuje raspored održanog turnira te najbolje takmičare i završetak turnira.

*testiraj_planiranje_turnira : –
planiraj_turnir.*



Figure 21: Plan turnira

Zaključak:

Ovaj kod za organizaciju Taekwondo turnira je sveobuhvatan i sadrži ključne elemente kao što su definisanje učesnika, disciplina, godina starosti, rezultata i pronalaženje najboljih boraca. Također, kod omogućava da se planira turnir sa pauzom. Pored toga, također sadrži testove za provjeru unosa takmičara i rezultata kao i provjere da takmičar ne može imati više rezultata u istoj disciplini kao i to da ne može biti više od jedne različite starosne kategorije.

Svaki takmičar je definisan njegovim imenom, godinama, težinom i pojasevima (zvanjem) dok su discipline jasno definisane. Kategorije mlađih uzrasta su definisane maksimalnim brojem godina za svaku kategoriju. Implementirana je funkcionalnost koja pronalazi odgovarajuće starosnim kategorijama za takmičare koji se ne mogu nalaziti u dvjema različitim kategorijama. Rezultati takmičara se definišu na osnovu različitih disciplina a funkcionalnost koja pronalazi sve najbolje borce po kategorijama materijalizovana je također. Plan turnira opisan je detaljno s vremenskim rasporedom aktivnosti navedenim po svakoj aktivnosti koje obuhvataju pauzu i ceremoniju dodjele medalja.

Također u kodu postoje neki nedostaci koji bi bili dobri da se analiziraju i unaprijede. To obuhvata ograničenja u interaktivnosti unosu takmičara i rezultata, nedovoljna fleksibilnost u trajanju mečeva te nedostatak pouzdanosti i efikasnosti pri upravljanju podataka o borcima i njihovim postignućima.

5 Problem 4: Organizacija putovanja

Danas nam se pruža mnogo prilika za putovanje širom svijeta. Imamo pristup internetu te samim tim možemo rezervirati hotele, letove i posjetu raznim turističkim mjestima ili ići putem agencije koja nudi organizaciju odlaska na željenu lokaciju. Međutim, željena lokacija nam najčešće predstavlja najveći problem. Pretpostavimo da smo ove godine, nakon što smo skupili novac, odlučili otići na putovanje, sami ili sa porodicom ili sa prijateljima. Imamo mnogo uslova i želimo da na osnovu datih želja odaberemo destinaciju koja će nam pružiti sve ono što tražimo. Mi ćemo riješiti ovaj problem koristeći se prologom. Napraviti ćemo svoju aplikaciju te dati kratke upute turistu.

Odabrali smo popularne i raznovrsne lokacije širom svijeta. Kako bi ispitali zahtjeve, postavljamo pitanja sa mogućim odgovorima da/ne. Na osnovu toga ćemo određivati putne lokacije.

Definisali smo kategorije pitanja te formulisali i sama pitanja, sa da/ne mogućim odgovorima na postavljeno pitanje. Nakon formuliranih pitanja postaviti ćemo podatke o agencijama. Agencije u našoj bazi podataka su na području Bosne i Hercegovine te na području Hrvatske. Svakoj agenciji smo dodijelili posebnu destinaciju, te imamo destinacija koliko imamo raspoloživih agencija.

```
requirement(Turist,odmor) : -verify(Turist, "Odmor [da|ne] ?", odmor).
requirement(Turist,zabava) : -verify(Turist, "Zabava [da|ne] ?", zabava).
requirement(Turist,poznanstvo) : -verify(Turist, "Poznanstva [da|ne] ?", poznanstvo).
requirement(Turist,kultura) : -verify(Turist, "Kultura [da|ne] ?", kultura).
requirement(Turist,romantika) : -verify(Turist, "Romantika [da|ne] ?", romantika).
requirement(Turist,toplota) : -verify(Turist, "Topla mjesta [da|ne] ?", toplota).
requirement(Turist,hladnoca) : -verify(Turist, "Hladnoca [da|ne] ?", hladnoca).
requirement(Turist,izlazak) : -verify(Turist, "Izlasci [da|ne] ?", izlazak).
requirement(Turist,iskustvo) : -verify(Turist, "Iskustva [da|ne] ?", iskustvo).
requirement(Turist,ekonomicnost) : -verify(Turist, "Budžet [da|ne] ?", ekonomicnost).
requirement(Turist,luksuz) : -verify(Turist, "Luksuz [da|ne] ?", luksuz).
requirement(Turist,porodica) : -verify(Turist, "Porodica [da|ne] ?", porodica).
requirement(Turist,prijatelji) : -verify(Turist, "Prijatelji [da|ne] ?", prijatelji).
requirement(Turist,partner) : -verify(Turist, "Partner [da|ne] ?", partner).
requirement(Turist,ljubimac) : -verify(Turist, "Ljubimci [da|ne] ?", ljubimac).
requirement(Turist,fizicki_problemi) : -verify(Turist, "Problemi [da|ne] ?", fizicki_problemi).
requirement(Turist,daljina) : -verify(Turist, "Daljina [da|ne] ?", daljina).
requirement(Turist,kraca_putovanja) : -verify(Turist, "Blizina [da|ne] ?", kraca_putovanja).
```

<i>agency(panturist).</i>	<i>assigned_to(panturist, pariz).</i>
<i>agency(petrol_travel).</i>	<i>assigned_to(petrol_travel, rim).</i>
<i>agency(atlantis_travel).</i>	<i>assigned_to(atlantis_travel,</i> <i>krakov).</i>
<i>agency(kozmopolit).</i>	<i>assigned_to(kozmopolit, shang-</i> <i>hai).</i>
<i>agency(jumbo_travel).</i>	<i>assigned_to(jumbo_travel,</i> <i>rosario).</i>
<i>agency(ulix_travel).</i>	<i>assigned_to(ulix_travel, sal-</i> <i>vador).</i>
<i>agency(katarina_line).</i>	<i>assigned_to(katarina_line,</i> <i>moskva).</i>
<i>agency(gulliver_travel).</i>	<i>assigned_to(gulliver_travel, am-</i> <i>sterdam).</i>
<i>agency(kompas_travel).</i>	<i>assigned_to(kompas_travel,</i> <i>kazablanka).</i>
<i>agency(adriatic_travel).</i>	<i>assigned_to(adriatic_travel,</i> <i>helsinki).</i>
<i>agency(fibula_air_travel).</i>	<i>assigned_to(fibula_air_travel,</i> <i>varadero).</i>
<i>agency(sarajevo_insider).</i>	<i>assigned_to(sarajevo_insider,</i> <i>chicago).</i>

Imamo bazu podataka o agencijama i destinacijama određenih za svaku agenciju posebno. Kako imamo različite gradove moramo navesti i pojedinačne hotele, odnosno nazive hotela za željene gradove. Nakon toga povezujemo gradove i imena država u kojoj se svaki grad nalazi:

<i>hotel(hotel_piazza, italija).</i>	<i>place(italija, rim).</i>
<i>hotel(hotel_amour, francuska).</i>	<i>place(francuska, pariz).</i>
<i>hotel(hotel_playa, argentina).</i>	<i>place(argentina, rosario).</i>
<i>hotel(hotel_freeze, poljska).</i>	<i>place(poljska, krakov).</i>
<i>hotel(hotel_cin, kina).</i>	<i>place(kina, shanghai).</i>
<i>hotel(hotel_brazio, brazil).</i>	<i>place(brazil, salvador).</i>
<i>hotel(hotel_raus, rusija).</i>	<i>place(rusija, moskva).</i>
<i>hotel(hotel_navigli, holandija).</i>	<i>place(holandija, amsterdam).</i>
<i>hotel(hotel_salah, maroko).</i>	<i>place(maroko, kazablanka).</i>
<i>hotel(hotel_yankees, amerika).</i>	<i>place(amerika, chicago).</i>
<i>hotel(hotel_fish, finska).</i>	<i>place(finska, helsinki).</i>
<i>hotel(hotel_beleza, kuba).</i>	<i>place(kuba, varadero).</i>

Svaka lokacija koja se nalazi u našoj bazi podataka je opisana kategorijama pitanja specificiranih za svaku državu posebno. Ukoliko nijedna od specifičnih pravila za određivanje zemlje nije zadovoljena ispisujemo grešku, odnosno da se

pravilo tretira kao pogrešno. Pogledajmo sljedeći dio našeg koda:

```
country(Tourist,italija) :-  
    requirement(Tourist,romantika),  
    requirement(Tourist,poznanstvo),  
    requirement(Tourist,ljubimac),  
    requirement(Tourist,ekonomicnost),  
    requirement(Tourist,kraca_putovanja),  
    requirement(Tourist,fizicki_problemi),  
    requirement(Tourist,zabava),!.  
country(Tourist,francuska) :-  
    requirement(Tourist,kultura),  
    requirement(Tourist,poznanstvo),  
    requirement(Tourist,kraca_putovanja),  
    requirement(Tourist,romantika),  
    requirement(Tourist,ekonomicnost),  
    requirement(Tourist,zabava),!.  
:  
country("Nemazemlje").
```

Provjera odgovora turista:

```
verify(Turist, Question, Fact) :-  
    print_formatted(Question, black),  
    read(Response),  
    normalize_response(Response, NormalizedResponse),  
    (NormalizedResponse == da -> assertz(yes(Turist, Fact));  
     NormalizedResponse == ne -> assertz(no(Turist, Fact));  
     verify(Turist, Question, Fact)).  
  
normalize_response(Response, NormalizedResponse) :-  
    string_lower(Response, LowercaseResponse),  
    (LowercaseResponse = "da" -> NormalizedResponse = da;  
     LowercaseResponse = "ne" -> NormalizedResponse = ne;  
     NormalizedResponse = invalid).
```

Sada pišemo samu srž našega koda, odnosno nakon što završimo analizu odgovora ispisujemo prijedloge turistu idealne lokacije na osnovu njegovih zahtjeva i želja:

```
travel(Turist, Drzava, Grad, Agencija, Hotel) :-  
    country(Turist, Drzava),  
    place(Drzava, Grad),
```

```

agency(Agencija),
assigned_to(Agencija, Grad),
hotel(Hotel, Drzava),
nl,
write("Idealna destinacija za vas je: "), nl, nl, nl,
sleep(0.3),
print_formatted(Drzava, yellow), nl, nl, nl,
sleep(0.3),
write("Idealni grad za vas je: "), nl, nl, nl,
sleep(0.3),
print_formatted(Grad, yellow), nl, nl, nl,
sleep(0.3),
write("Savjetujemo vam da putujete sa agencijom: "), nl, nl, nl,
sleep(0.3),
print_formatted(Agencija, yellow), nl, nl, nl,
sleep(0.3),
write("Savjetujemo vam da odsjete u hotelu: "), nl, nl, nl,
sleep(0.3),
print_formatted(Hotel, yellow), nl, nl, nl,
sleep(0.3),
write("Ugodan boravak vam želimo :)"), nl, nl, nl.
start_questions :-
    nl, nl,
    write("Kako se zovete?"),
    read(Ime),
    travel(Ime, Drzava, Grad, Agencija, Hotel).

```

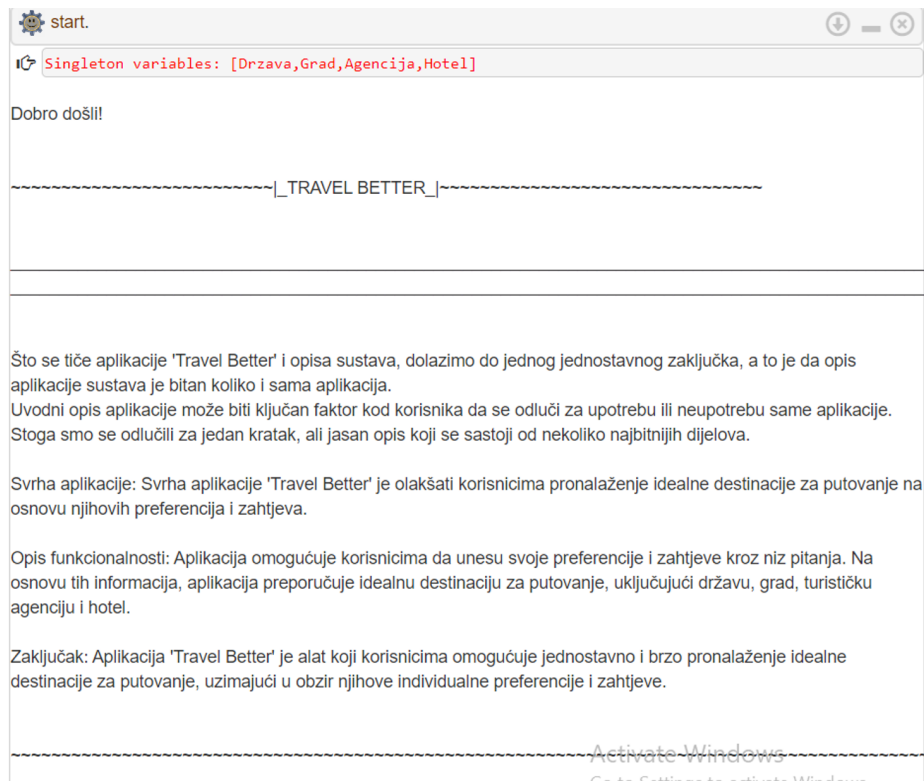


Figure 22: Pokretanje programa naredbom start.

Sada možete započeti korištenje aplikacije!

Odgovorite na sljedeća pitanja kako bismo vam pronašli idealnu destinaciju za vaše putovanje.

Molimo vas da na pitanja odgovarate sa 'da' ili 'ne'.

Da li ste spremni za početak? (da/ne)

da

Kako se zovete?

Nedzmija

Da li ste u potrazi za partnerom [da|ne] ?

ne

Da li ste otvoreni za sklapanje novih poznanstava [da|ne] ?

da

Putujete li sa jednim ili više kućnih ljubimaca [da|ne] ?

ne

Želite li što manje novca potrošiti [da|ne] ?

da

Preferirate li bliža putovanja [da|ne] ?

da

Imate li nekih fizičkih problema [da|ne] ?

ne

Da li ste osoba koja voli zabavu [da|ne] ?

da

Figure 23: Unos informacija putnika

```

Idealna destinacija za vas je:

italija

Idealni grad za vas je:

rim

Savjetujemo vam da putujete sa agencijom:

petrol_travel

Savjetujemo vam da odsjete u hotelu:

hotel_piazza

Ugodan boravak vam želimo :)

true
false

```

Figure 24: Konačni izlaz programa

Zaključak: Implementacijom problema organizacije putovanja u Prologu smo uspješno razvili sistem koji omogućava preporuku putovanja na osnovu korisnikovih preferencija. Kod je baziran na predikatima za verifikaciju uslova i postavljanju pitanja, čime smo omogućili interakciju sa turistom i prilagodljivu organizaciju putovanja. Bazirali smo naš kod na pitanjima i rasporedu putem odgovorene ankete, ali postoji i prostor za proširenje i unapređenje dodavanjem novih kriterija i opcija čime bi sistem postao fleksibilniji i korisniji.

6 Izvori

- <https://builtin.com/software-engineering-perspectives/prolog>
- https://www.tutorialspoint.com/prolog/prolog_introduction.htm

7 Izvještaj

Uvodni dio kao i problem 1 uradila je Zerina Pandža, problem 2 je uradio Enis Adilović, problem 3 je uradio Faris Aljić, problem 4 je uradila Nedžmija Karić, dok članica grupe Fadila Begić nije učestvovala u izradi ove zadaće.